Logan Hunt (solo, team leader)

10/11/2026

CMPSC 448 Section 001

Who Wrote It? Identifying LLMs from Their Responses, Midterm Project

Problem Definition and Dataset

The question this project asks is whether you can tell the responses of different large language models apart. As a classification problem, the input is a model's response (and sometimes the prompt that created that response) and the output is which LLM wrote it. I trained two neural networks to solve this problem, a convolutional neural network and a long short-term memory network which is a recurrent neural network.

The dataset I used is the LMSYS Chatbot Arena Conversations dataset on Hugging Face. Between April and June 2023, users on Chatbot Arena would send a single prompt and then compare the outputs of two different LLMs, voting on which was better. This meant that each row was a prompt, LLM outputs, and labels of which LLMs the outputs came from. The dataset included conversations of multiple turns, so first I filtered down to conversations that had just one turn, one prompt for two LLM outputs. I then kept only English conversations and ones that weren't flagged by OpenAI's moderation. I had made the filter before checking if there was flagged data to begin with, but upon checking there appears to not be any. Then, since each row has two answers, I split them into two examples, one per model.

The dataset included many models, but I filtered them down to four groups of similar models to make classification simpler:

Group	Models	Examples
GPT	gpt-4, gpt-3.5-turbo	7,047
Claude	claude-v1, claude-instant-v1	5,206
Llama	vicuna-13b, koala-13b, alpaca-13b, llama-13b	14,013
PaLM	palm-2	2,325

At this point, 14,013 out of 28,591 examples were from Llama versions, so a model could get about 50% accuracy just by always guessing Llama. To stop that, I randomly sampled the minimum, 2,325, from each of them so that the examples were balanced. For splitting, I put 70% into training, 15% into validation, and 15% into testing. Prompts were not unique, and having the same prompt in the training set and validation or testing sets could mean a model could memorize the answer instead of learning it, so I split the data by prompt, and the splitting script checks that no prompt appears in more than one set. By the end, the split was about 1,600 training, 350 validation and 350 test examples per group.

Models and Training

First, the text has to be turned into numbers that the models can process. I wrote a tokenizer that lowercases the text and then splits it with regex into three types of token, a newline, a word, or punctuation. The vocabulary is then built using the tokens from the training set, with three additional tokens, <pad> for padding so that every sequence is the same length, <unk> for words not found in the vocabulary, and <sep> to separate the prompt and response when both are used for training.

I kept newlines and punctuation on purpose, since formatting like bullets, bold text, and lists could be part of what makes each LLM recognizable. I also kept only tokens that showed up at least twice in training, and based on the median response being 171 tokens long, I capped each sequence to 256 tokens, using <pad> to fill up sequences shorter than that.

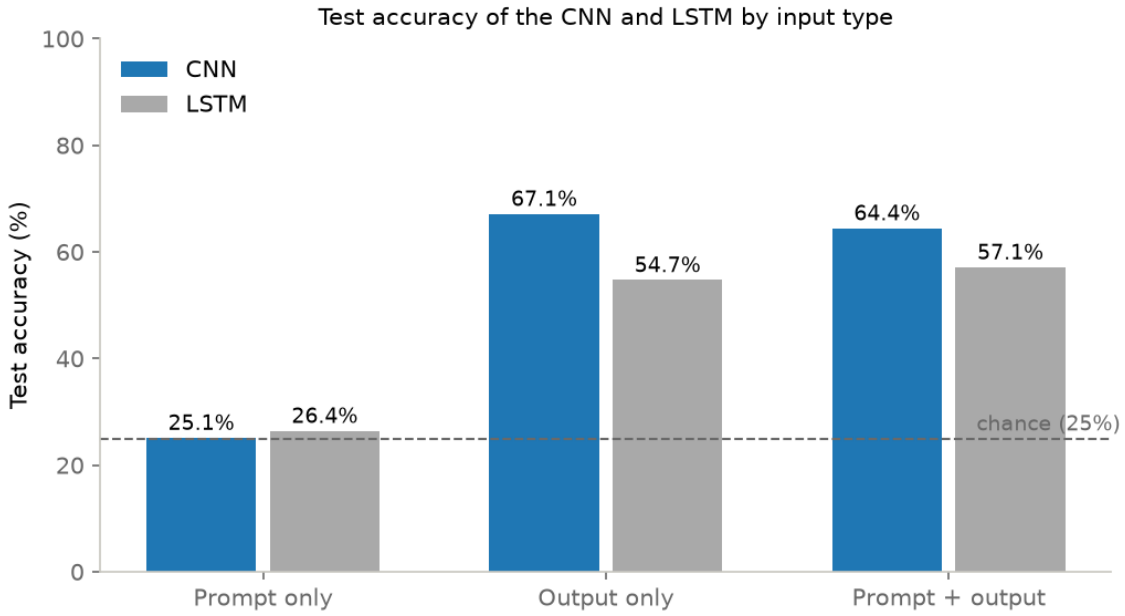
The CNN turns each token into a vector of 128 numbers with an embedding layer, then runs 128 filters over the text, each covering 5 tokens at a time. After going through ReLU I take each filter's maximum over the whole sequence, so I get one number per filter indicating how strongly the filter matched the sequence. Those 128 numbers go through dropout and a linear layer that outputs a logit, or score for each of the LLM groups.

The LSTM was written modeling the equations from class. The forget gate decides how much of the old cell state to keep, the input gate decides how much of the new candidate to add, and the output gate decides how much of the cell state becomes hidden state. A problem I had to account for was padding, I believe because short responses were padded with many <pad>'s, the hidden state would represent all those <pad>'s instead of the meaningful data, so instead if the token was a <pad> then the cell state and hidden state would not be updated. This way, the last hidden state corresponds to the last actual token in the sequence instead of possibly a <pad>, and it goes through a dropout and linear layer like the CNN.

Both the LSTM and CNN have about 3.1 million parameters, with most of them coming from the embedding tables. Both were trained the same way, using cross-entropy loss, and the Adam optimizer implementation built into PyTorch at a learning rate of 0.001, batches of size 64 and 10 epochs. I tracked each epoch, saving the weights every time a new best accuracy on the validation set was reached. After training I evaluated the best weights on the test set.

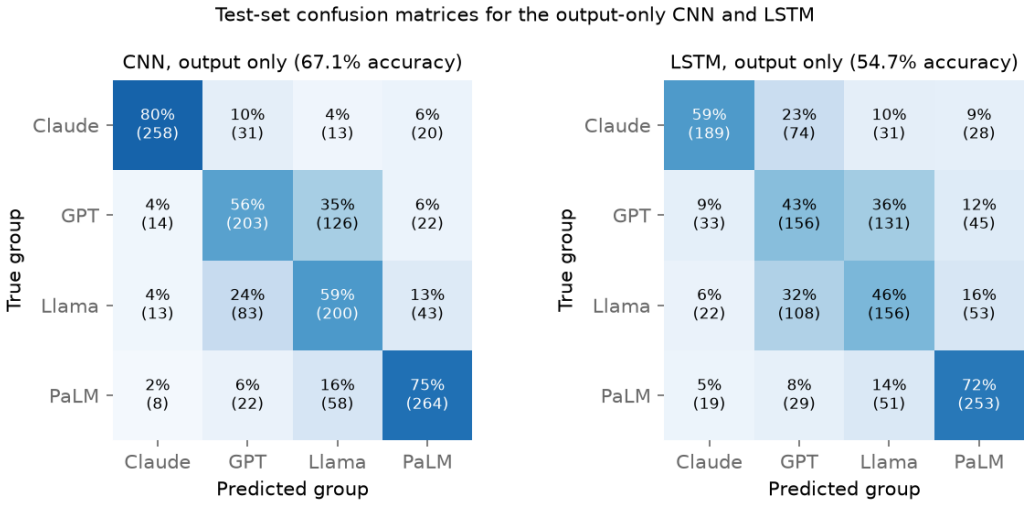
Results

Below are the models' accuracy and macro-F1, the average F1 score over each group:



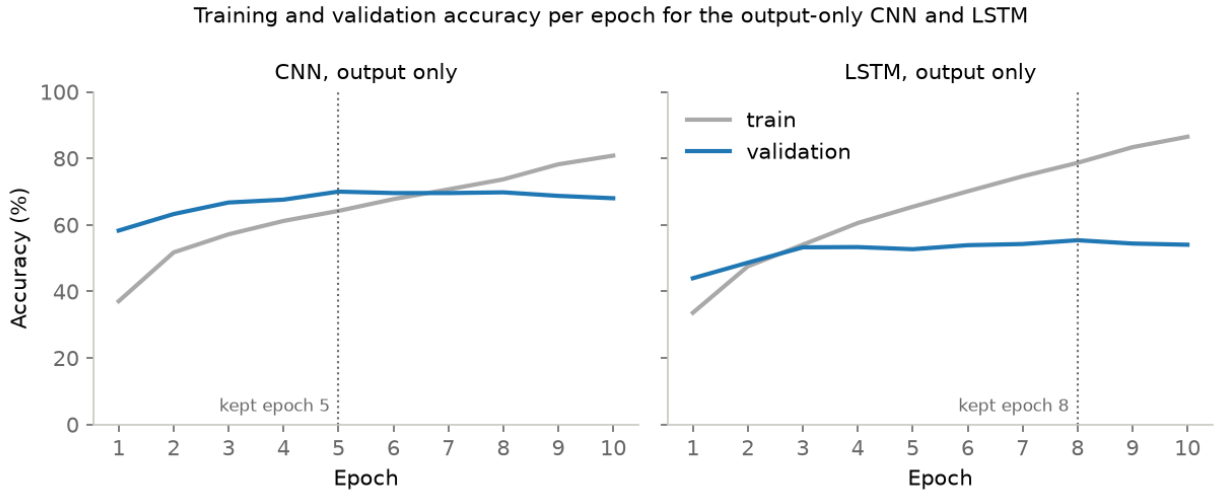
Model (macro-F1)	Prompt only	Output only	Prompt + output
CNN	0.233	0.678	0.648
LSTM	0.259	0.551	0.569

Both models can identify which LLM group the data belongs to based on output only, with the CNN doing noticeably better than the LSTM. Claude was recognized correctly most often, with 80% recall, then PaLM at 75%, Llama with 59%, and the GPT group at 56%. Most mistakes were Llama and GPT getting mistaken for each other.



When using only the prompt for training, the accuracy was not significantly better than just guessing. Adding the prompt before the response lowered the CNN's accuracy and slightly raised the LSTM's, so the prompt does not significantly help in identifying the LLM, though this could be due to the nature of the dataset.

Both models overfit during training. Their training accuracies both increase above their validation accuracies before the last epoch. The overfitting didn't affect results, however, since only the weights from the epoch with the highest validation accuracy (the best one) are kept.

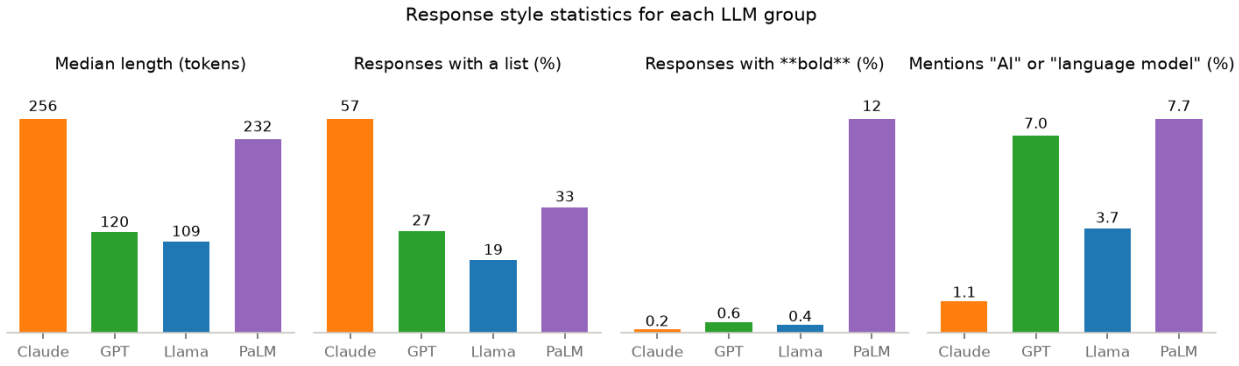


Analysis

During Chatbot Arena, users would type their prompt before they knew which models would be answering, which means the prompts could carry almost no information that identified which model answered it. This explains why using only the prompt in training resulted in about 25% accuracy, not significantly better than just guessing. That being said, during training the CNN still reached 61% accuracy, and the LSTM reached 73% accuracy, but this was likely just the models memorizing the training prompts instead of learning anything valuable, since the validation accuracies never got much higher than 25%.

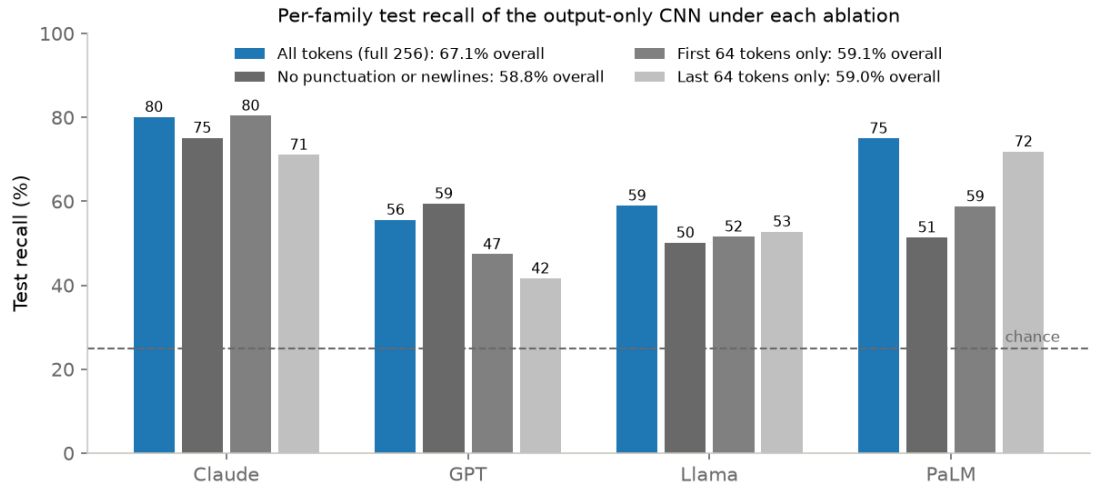
Llama and GPT were most confused for each other likely because of how the Llama models were trained. Vicuna and Koala were trained on ChatGPT conversations, and Alpaca was trained on answers from OpenAI's text-davinci-003, so they likely learned to imitate many of the same patterns ChatGPT exhibited.

I looked at many different style and vocabulary statistics for each of the groups, and specifically for tokens that were distinct and were used by one group significantly more than the others.



Claude and PaLM write significantly longer than the GPT or Llama models. Claude uses a list more often than the other models, and also uses the character "•" for its lists, with it appearing in 23% of its responses versus less than 1% for every other group. PaLM uses two asterisks to make bold text more than ten times as often as the other models, and it also reuses the same refusal message of "I'm not able to help with that, as I'm only a language model. If you believe this is an error, please send us your feedback." The GPT models had very distinctive words like "ensuring", "crucial", and "essential", and interestingly a few of the Llama responses had HTML tags, likely due to the models being trained on scraped ShareGPT conversations.

I retrained the CNN three times with different amounts of information removed to see which signals were being used the most to identify each group. I trained it with word tokens only, with only the first 64 tokens, and with only the last 64 tokens.



PaLM's fingerprint seems to be based on its formatting more than the other groups' fingerprints are. When formatting elements (punctuation and newline characters) were removed, PaLM had the largest fall in test recall. Claude was the most consistent, and had no loss in recall when using only the first 64 tokens, likely because it starts its responses more consistently than the other groups. Similarly, PaLM's recall for the last 64 tokens only lowered slightly, likely because of the refusal message it reuses. Overall, the signals are both structural and linguistic, with the strength of each depending on the group.

The CNN was consistently better than the LSTM. This is likely because most of the identifying features for each of these groups are small patterns that can show up anywhere in the sequence, which is specifically what the max-pool step finds from all the filter outputs. In the LSTM's case, it has to find these patterns and carry them through every hidden state regardless of where the pattern is found, so patterns found early on can be lost.

Lessons and Experience

This was the first time I used PyTorch. My first full run had two bugs that didn't cause errors, so they were hard to find. I forgot to apply the dropout in the CNN, and in the LSTM I was using the output gate to compute the candidate instead of the candidate weight. Fixing the LSTM raised its output-only accuracy from 52.2% to 54.7%, but fixing the dropout in the CNN actually lowered its test accuracy from 71.6% to 67.1%, so even though it wasn't overfitting as much, its score still went down.

I had figured that implementing the models and loop would be the biggest challenge, but those steps were relatively straightforward when I followed the algorithms from the class slides. I learned that most of the work is in processing and analyzing the data. In this case specifically, I had to do a lot of processing to get the data into the form I needed, rows containing at least an LLM label, a prompt, and an output. Then, that data had to be split by prompt, otherwise a model could memorize repeated prompts and have a high training accuracy but low test accuracy. The most interesting findings, like the HTML in Llama responses, also came from analyzing the data rather than the results of the models.

I used Claude throughout this project to tutor me in PyTorch, to review and debug my code, and to help interpret data analysis and model output. I wrote, ran, and checked all the code in the repository myself.

References

Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances In Neural Information Processing Systems. Dataset: https://huggingface.co/datasets/lmsys/chatbot_arena_conversations

LMSYS (2023, March 30). Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90% ChatGPT Quality. https://www.lmsys.org/blog/2023-03-30-vicuna/

Geng, X., et al. (2023). Koala: A dialogue model for academic research. The Berkeley Artificial Intelligence Research Blog. http://bair.berkeley.edu/blog/2023/04/03/koala/

Taori, R., et al. (2023). Alpaca: A Strong, Replicable Instruction-Following Model. Stanford CRFM. https://crfm.stanford.edu/2023/03/13/alpaca.html